

Задание 1: Анализ требований

Описание задания:

Вам попал на руки фрагмент технического задания на разработку функционала "Корзина" для интернет-магазина "Петрушка Зеленая". В ТЗ есть несколько логических противоречий, недочетов и неполных мест.

Раздел ТЗ: Функционал корзины

№	Требование
1	Пользователь может добавить в корзину от 1 до 10 единиц одного товара.
2	Пользователь может изменить количество каждого товара в корзине не менее, чем до 1-го. Для удаления товара из корзины используется отдельная кнопка.
3	В корзине может находиться не более 5 различных товаров.
4	Суммарное количество всех товаров в корзине не может превышать 20 штук.
5	Товары в корзине могут быть разные.
6	При попытке добавить товар, превышающий лимиты, система показывает сообщение: "Лимит корзины превышен".
7	Цена на продукт фиксируется на момент добавления в корзину и не меняется.
8	На странице корзины отображается список товаров, их количество, цена за единицу и общая стоимость позиции.
9	Если пользователь уменьшает количество товара до 0, товар удаляется из корзины.
10	В корзине может быть реклама других продуктов.
11	Реклама товаров в корзине должна быть каждый будний день по утрам и вечерам.
13	Если цена на товар изменилась в каталоге, система должна автоматически обновить ее в корзине у всех пользователей.

Что нужно сделать:

- Найдите и перечислите все логические противоречия и недочеты в предоставленном ТЗ.
Объясните, в чем заключается проблема для каждого пункта.
- Предложите конкретные исправления для устранения этих противоречий - напишите свою версию этого фрагмента ТЗ, которая должна быть логически завершенной и непротиворечивой.

3. Какие уточняющие вопросы вы бы задали продукт-менеджеру или бизнес-заказчику по этому ТЗ?

Решение

1. Анализ противоречий и недочетов

№	Требование	Тип проблемы	Описание проблемы
1	Пользователь может добавить в корзину от 1 до 10 единиц одного товара.	Неоднозначность	Неясно, относится ли ограничение к одному действию добавления товара или к итоговому количеству одной товарной позиции в корзине. Например, непонятно, может ли пользователь дважды добавить один и тот же товар по 10 единиц, если суммарный лимит корзины составляет 20 товаров.
2	Пользователь может изменить количество каждого товара в корзине не менее, чем до 1-го. Для удаления товара из корзины используется отдельная кнопка.	1) Неполное описание 2) Противоречие 3) Нарушение атомарности	1) Указана только нижняя граница изменения количества товара, но не указан верхний предел. 2) Требование противоречит п.9: в п.2 количество нельзя уменьшить ниже 1, а в п.9 товар удаляется при уменьшении количества до 0. 3) В одном требовании описаны два разных действия: изменение количества товара и удаление товара. Лучше разделить их на отдельные требования.
3	В корзине может находиться не более 5 различных товаров.	-	-
4	Суммарное количество всех товаров в корзине не может превышать 20 штук.	-	-
5	Товары в корзине могут быть разные.	Избыточность	Требование дублирует смысл п.3, где уже указано, что в корзине могут находиться различные товары, и задан лимит на их количество.

№	Требование	Тип проблемы	Описание проблемы
6	При попытке добавить товар, превышающий лимиты, система показывает сообщение: "Лимит корзины превышен".	Неполное описание	Сообщение слишком общее, так как в ТЗ описано несколько разных лимитов: по количеству одного товара, по количеству различных товаров и по общему количеству товаров в корзине. Пользователю будет непонятно, какой именно лимит превышен. Также не указано, добавляется ли допустимое количество или добавление полностью блокируется.
7	Цена на продукт фиксируется на момент добавления в корзину и не меняется.	Противоречие	Требование противоречит п.13, где указано, что при изменении цены в каталоге система должна автоматически обновить цену товара в корзине у всех пользователей. Нужно уточнить бизнес-логику: фиксируется ли цена при добавлении в корзину или актуализируется до момента оформления заказа/оплаты.
8	На странице корзины отображается список товаров, их количество, цена за единицу и общая стоимость позиции.	Неполное описание	Описание страницы корзины неполное. Не указаны кнопка удаления товара, итоговая сумма корзины, возможный рекламный блок из п.10 и кнопки изменения количества единиц товара, удаления всех или выбранных позиций, оформления заказа.
9	Если пользователь уменьшает количество товара до 0, товар удаляется из корзины.	Противоречие	См п.2
10	В корзине может быть реклама других продуктов.	Неполное описание	Не описано, где именно отображается реклама, сколько рекламных блоков или товаров показывается, по каким правилам подбираются рекламируемые товары и можно ли скрыть рекламный блок. Также требование связано с п.11 и должно быть описано вместе с правилами показа рекламы.

№	Требование	Тип проблемы	Описание проблемы
11	Реклама товаров в корзине должна быть каждый будний день по утрам и вечерам.	1) Неоднозначность 2) Неполное описание	1) Не указано, что считается утром и вечером: нужны конкретные временные интервалы и часовой пояс. 2) Не описаны правила показа рекламы: место отображения, количество рекламных блоков или товаров в блоке, частота смены контента, источник товаров для рекламы и поведение рекламы в выходные дни.
13	Если цена на товар изменилась в каталоге, система должна автоматически обновить ее в корзине у всех пользователей.	1) Противоречие 2) Нарушение нумерации	1) См п.7 2) В списке требований пропущен пункт 12: после п.11 сразу идет п.13. Это может вызвать путаницу при обсуждении и ссылках на требования.

Общий вывод по исходным требованиям

В исходном фрагменте ТЗ выявлены следующие проблемы:

Тип проблемы	Требования
Логические противоречия	2-9, 7-13
Неоднозначные формулировки	1, 11
Неполное описание	2, 6, 8, 10, 11
Нарушение атомарности	2
Избыточность	5
Формальный недочет	отсутствует п.12

Также есть замечания к структуре требований: требования приведены единым списком без группировки по функциональным блокам: лимиты корзины, добавление товара, изменение количества, удаление, отображение корзины, цены и реклама описаны вперемешку. Из-за этого требования сложнее анализировать, проверять на полноту и использовать при разработке.

Дополнительно - отсутствует единообразие терминологии: в разных пунктах используются термины "товар", "продукт", "единица товара", "позиция". Для исключения разночтений стоит ввести глоссарий терминов и явно определить, что означает каждый термин. Например: "товар" - карточка товара из

каталога, "товарная позиция" — строка корзины по конкретному товару, "единица товара" - количество товара внутри товарной позиции.

2. Исправленная версия требований

2.1. Глоссарий

Термин	Описание
Товар	Карточка товара из каталога интернет-магазина.
Товарная позиция	Строка корзины, соответствующая одному конкретному товару.
Единица товара	Количество товара внутри товарной позиции.
Корзина	Набор товарных позиций, выбранных пользователем до оформления заказа.

2.2. Функциональные требования

Лимиты корзины

CART-LIMIT-001. Корзина пользователя может содержать не более 5 различных товарных позиций.

CART-LIMIT-002. Общее количество единиц товаров в корзине пользователя не должно превышать 20.

CART-LIMIT-003. Одна товарная позиция в корзине пользователя может содержать от 1 до 10 единиц товара включительно.

Добавление товара в корзину

CART-ADD-001. Пользователь может добавить товар в корзину со страницы товара или из каталога.

CART-ADD-002. При добавлении товара, которого еще нет в корзине, система создает новую товарную позицию.

CART-ADD-003. При повторном добавлении товара, который уже есть в корзине, система увеличивает количество существующей товарной позиции.

CART-ADD-004. Система не должна добавлять товар в корзину, если после добавления будет превышен хотя бы один из лимитов, описанных в CART-LIMIT-001, CART-LIMIT-002 или CART-LIMIT-003.

Изменение количества товара

CART-QTY-001. Пользователь может изменить количество единиц товара в товарной позиции в пределах от 1 до 10 включительно.

CART-QTY-002. Если при изменении количества будет превышен общий лимит корзины в 20 единиц товаров, система не должна применять изменение.

Удаление товара из корзины

CART-DELETE-001. Для каждой товарной позиции в корзине отображается кнопка "Удалить".

CART-DELETE-002. При нажатии на кнопку "Удалить" система удаляет товарную позицию из корзины.

CART-DELETE-003. После удаления товарной позиции система пересчитывает итоговую стоимость корзины.

Сообщения об ошибках и ограничениях

CART-VALIDATION-001. Если пользователь пытается добавить больше 10 единиц одного товара, система не добавляет товар и показывает сообщение: "В корзину можно добавить не более 10 единиц одного товара".

CART-VALIDATION-002. Если пользователь пытается добавить больше 5 различных товарных позиций, система не добавляет товар и показывает сообщение: "В корзине может быть не более 5 разных товаров".

CART-VALIDATION-003. Если пользователь пытается добавить товар сверх общего лимита в 20 единиц товаров, система не добавляет товар и показывает сообщение: "Общее количество товаров в корзине не может превышать 20".

Отображение страницы корзины

CART-VIEW-001. На странице корзины отображается список товарных позиций.

CART-VIEW-002. Для каждой товарной позиции отображаются: наименование товара, количество единиц товара, цена за единицу товара, общая стоимость товарной позиции и кнопка "Удалить".

CART-VIEW-003. Общая стоимость товарной позиции рассчитывается по формуле: количество единиц товара × цена за единицу товара.

CART-VIEW-004. На странице корзины отображается итоговая стоимость всех товарных позиций.

CART-VIEW-005. На странице корзины отображается кнопка "Оформить заказ".

Цена товара в корзине

CART-PRICE-001. Цена товара в корзине должна соответствовать актуальной цене товара в каталоге.

CART-PRICE-002. Если цена товара изменилась в каталоге, система автоматически обновляет цену соответствующей товарной позиции в корзине пользователя.

CART-PRICE-003. Финальная цена товара фиксируется в момент оформления заказа.

Рекламный блок в корзине

CART-ADS-001. В корзине может отображаться рекламный блок с товарами из каталога.

CART-ADS-002. Рекламный блок отображается на странице корзины под списком товарных позиций.

CART-ADS-003. В рекламном блоке отображается не более 3 товаров.

CART-ADS-004. Рекламный блок отображается по будним дням в следующие временные интервалы: с 09:00 до 12:00 и с 18:00 до 21:00 по московскому времени.

CART-ADS-005. В выходные дни рекламный блок не отображается.

CART-ADS-006. Товары для рекламного блока выбираются из активных товаров каталога, доступных для покупки.

CART-ADS-007. Товары, которые уже находятся в корзине пользователя, не должны отображаться в рекламном блоке.

2.3. Принятые допущения

В исправленной версии требований выбран следующий вариант бизнес-логики:

- удаление товарной позиции выполняется только через отдельную кнопку "Удалить";
- изменение количества товара доступно в диапазоне от 1 до 10 включительно;
- цена товара в корзине актуализируется при изменении цены в каталоге;
- финальная цена фиксируется в момент оформления заказа;
- рекламный блок описан как часть страницы корзины с конкретными правилами показа.

Требования сгруппированы по функциональным разделам. Идентификаторы требований сформированы по шаблону `CART-{SECTION}-{NUMBER}` , нумерация ведется отдельно внутри каждого раздела.

3. Уточняющие вопросы к продукт-менеджеру или бизнес-заказчику

Лимиты корзины

1. Ограничение "от 1 до 10 единиц одного товара" относится к итоговому количеству одной товарной позиции в корзине или только к одному действию добавления товара?
2. Что ещё должна делать система кроме оповещения, если пользователь пытается добавить количество товара сверх лимита: полностью отклонять действие или добавлять максимально допустимое количество?
3. Должны ли лимиты корзины быть одинаковыми для всех пользователей или могут зависеть от типа товара, региона, склада, пользователя или акции?
4. Нужно ли показывать пользователю заранее информацию о лимитах корзины, например "максимум 10 единиц одного товара"?

Добавление и изменение количества

1. Что должно происходить при повторном добавлении товара, который уже есть в корзине: увеличивать количество существующей товарной позиции или показывать сообщение, что товар уже добавлен?
2. Нужно ли проверять наличие товара на складе при добавлении в корзину и изменении количества?
3. Что делать, если пользователь добавил товар в корзину, но позже он стал недоступен для покупки?

Удаление товара

1. Нужно ли подтверждение перед удалением товарной позиции из корзины?
2. Должна ли быть возможность восстановить удаленный товар, например через сообщение "Товар удален. Отменить"?
3. Нужна ли возможность очистки всей корзины или групповое удаление выбранных товарных позиций?

Цена товара

1. В какой момент должна фиксироваться цена товара: при добавлении в корзину, при переходе к оформлению заказа или при оплате?

2. Что должно происходить, если цена товара изменилась, пока пользователь находится на странице корзины?
3. Нужно ли уведомлять пользователя об изменении цены в корзине?
4. Как поступать, если цена изменилась после оформления заказа, но до оплаты?

Отображение корзины

1. Какие элементы должны отображаться на странице корзины помимо списка товаров: итоговая сумма, кнопка оформления заказа, кнопка удаления, количество товаров, рекламный блок?
2. Нужно ли отображать изображение товара, название, вес/объем, скидку, старую цену или другую дополнительную информацию?
3. Нужно ли показывать пустое состояние корзины, если в ней нет товаров?
4. Какой текст и какие действия должны быть доступны пользователю в пустой корзине?

Сообщения об ошибках и ограничениях

1. Должны ли сообщения об ошибках быть общими или отдельными для каждого лимита?
2. Какие тексты сообщений нужно показывать при превышении лимита одного товара, лимита разных товаров и общего количества товаров?
3. Как именно показывать сообщения: toast-уведомлением, модальным окном, inline-сообщением рядом с товаром или другим способом?

Реклама в корзине

1. Что именно считается рекламой других продуктов: товары из каталога, акции, баннеры или персональные рекомендации?
2. Где должен отображаться рекламный блок на странице корзины?
3. Сколько товаров или баннеров должно отображаться в рекламном блоке?
4. Как выбирать товары для рекламы: случайно, по популярности, по категории товаров в корзине, по персональным рекомендациям или по правилам маркетинга?
5. Нужно ли исключать из рекламы товары, которые уже находятся в корзине?
6. Что считать "утром" и "вечером" для показа рекламы?
7. В какой таймзоне должны рассчитываться утро и вечер?
8. Что должно происходить с рекламным блоком в выходные дни?
9. Можно ли пользователю скрыть рекламный блок?

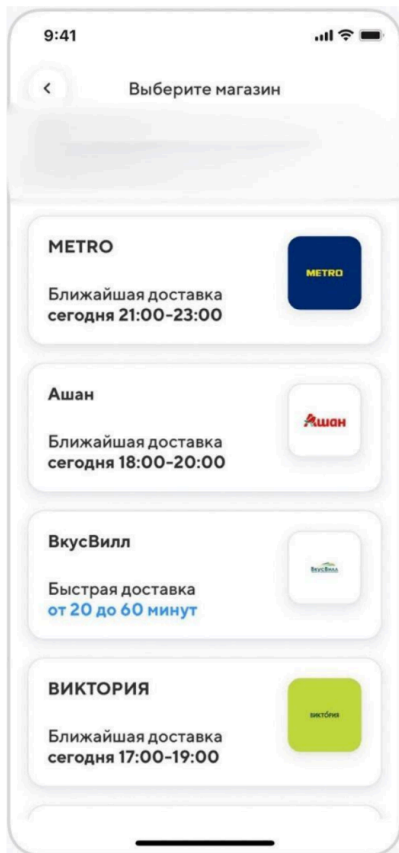
Технические и бизнес-ограничения

1. Корзина должна храниться только для авторизованных пользователей или также для гостей?
2. Если пользователь не авторизован, где должна храниться корзина: на устройстве, в браузере или на сервере?
3. Нужно ли синхронизировать корзину между несколькими устройствами пользователя?
4. Что должно происходить с корзиной после оформления заказа?
5. Сколько времени должна храниться неоформленная корзина?

Задание 2: Проектирование API

Описание:

Интернет-магазин "Петрушка Зеленая" преуспевает, расширяется и в мобильном приложении решили создать новый экран, который будет отображать магазины партнеров (см. макеты ниже).



Что нужно сделать:

1. Написать пример REST API запроса, который будет вызываться при переходе пользователя на данный экран.
2. Привести пример ответа этого REST API в соответствии с макетом. Формат - JSON. Учесть, что при клике на плашку магазина должен осуществляться переход по ссылке на внешний ресурс.

Решение

1. Анализ макета

Экран содержит:

- кнопку "Назад";

- заголовок экрана;
- список карточек магазинов-партнеров.

Кнопка "Назад" и заголовок относятся к интерфейсу мобильного приложения и не требуют отдельного API.

На макете отображается вертикальный список карточек магазинов. Элемент пагинации на экране не показан, но список может быть длинным, поэтому в API стоит предусмотреть постраничную загрузку данных. Это позволит мобильному приложению реализовать подгрузку следующей страницы при прокрутке списка.

Для отображения карточки магазина приложению нужны следующие данные:

- `id` - `string` . Уникальный идентификатор магазина-партнера.
- `name` - `string` . Название магазина для отображения на карточке.
- `logoUrl` - `string` . Ссылка для загрузки логотипа магазина. Логотип хранится во внешнем файловом хранилище, а приложение получает готовый URL для отображения изображения.
- `externalUrl` - `string` . Ссылка на внешний ресурс партнера, которая открывается при клике на карточку магазина.
- `deliveryType` - `string` . Тип доставки. Возможные значения: `time_slot` , `fast_delivery` .
- `deliveryLabel` - `string` . Текстовое описание типа доставки, например "Ближайшая доставка" или "Быстрая доставка".
- `deliveryValueText` - `string` . Текстовое значение доставки для отображения в интерфейсе, например "сегодня 21:00–23:00" или "от 20 до 60 минут". Визуальное оформление зависит от `deliveryType` .

2. Описание и принятые допущения

1. Метод вызывается при переходе пользователя на экран "Выберите магазин" и возвращает список активных магазинов-партнеров с постраничной пагинацией, доступных для выбранного города.
2. Предполагается, что экран рендерится на стороне мобильного приложения. Backend возвращает данные в формате JSON, а мобильное приложение самостоятельно формирует карточки магазинов по макету.
3. Список магазинов-партнеров является публичным и не требует обязательной авторизации.
4. Город может быть определен мобильным приложением:
 - из профиля авторизованного пользователя;
 - из ранее выбранного города;
 - по геолокации пользователя;
 - через экран ручного выбора города.
5. Если город не определен, мобильное приложение должно сначала запросить выбор города у пользователя и только после этого вызвать метод получения магазинов-партнеров.
6. В текущем решении список магазинов определяется по параметру `cityId` , так как в задании нет конкретики и мной выбран простой вариант. (В реальном приложении для более точного расчета доступных магазинов и интервалов доставки лучше использовать адрес доставки пользователя, район города или зону доставки.)

7. Backend рассчитывает ближайший доступный интервал доставки на момент обработки запроса с учетом таймзоны выбранного города.
8. Таймзона хранится на стороне backend и определяется по `cityId`.
9. В текущем варианте API возвращает готовый текст доставки в поле `delivery.valueText`, так как по заданию требуется отобразить данные на экране, а сортировка и фильтрация по доставке в макете и описании задания не указаны.
10. В рамках текущего задания сортировка, фильтрация и поиск по списку магазинов не описаны, поэтому магазины отображаются в том порядке, в котором они возвращены backend в массиве `stores`.
11. Логотип магазина передается в виде ссылки `logoUrl`. Отдельный API-запрос для получения логотипа не требуется.
12. Поле `externalUrl` в рамках данного решения содержит прямую ссылку на сайт партнера без дополнительных параметров для аналитики и без перенаправляющих сервисов.

3. REST API запрос

Краткое описание: Получение списка магазинов с постраничной пагинацией

Метод: GET

URL: `/api/v1/partner-stores?cityId={string}`

Структура запроса

- **Заголовки**
 - `Accept: application/json`
- **Query-параметры**
 - `cityId`
 - Тип: *string*
 - Обязательный: да
 - Значение по умолчанию: отсутствует
 - Описание: идентификатор выбранного города.
 - `limit`
 - Тип: *integer*
 - Обязательный: нет
 - Значение по умолчанию: 20
 - Описание: максимальное количество магазинов в ответе.
 - `offset`
 - Тип: *integer*
 - Обязательный: нет
 - Значение по умолчанию: 0
 - Описание: смещение для постраничной загрузки списка.

Пример запроса

GET https://{домен}/api/v1/partner-stores?cityId=moscow&limit=4&offset=0
Accept: application/json

Структура ответа

► Успешный ответ

HTTP status: 200 OK

Поля ответа:

1. pagination - object . Информация о постраничной загрузке списка.
 - limit - integer . Максимальное количество элементов в ответе.
 - offset - integer . Смещение текущей страницы.
 - total - integer . Общее количество найденных магазинов-партнеров.
 - hasMore - boolean . Признак наличия следующей страницы.
2. stores - array . Список магазинов-партнеров.

Для каждого найденного магазина возвращается:

- id - string . Уникальный идентификатор магазина-партнера.
- name - string . Название магазина для отображения на карточке.
- logoUrl - string . Ссылка для загрузки логотипа магазина.
- externalUrl - string . Ссылка на внешний ресурс партнера. При клике на карточку магазин открывается по этой ссылке.
- delivery - object . Информация о доставке.
 - type - string . Тип доставки. Возможные значения: time_slot , fast_delivery .
 - label - string . Текстовая подпись доставки.
 - valueText - string . Текстовое значение доставки для отображения в интерфейсе.

Пример ответа

```
{
  "serverTime": "2026-06-28T16:30:00+03:00",
  "timezone": "Europe/Moscow",
  "pagination": {
    "limit": 4,
    "offset": 0,
    "total": 16,
    "hasMore": true
  },
  "stores": [
    {
      "id": "2f4a6f4c-8e9b-4c4e-9f1a-8a9c2f0d3b11",
      "name": "METRO",
      "logoUrl": "https://{домен}/api/v1/files/store-logos/metro.png",
      "externalUrl": "https://metro.example.com",
      "delivery": {
        "type": "time_slot",
        "label": "Ближайшая доставка",

```

```

        "valueText": "сегодня 21:00-23:00"
    }
},
{
    "id": "7b1d9f2e-6c35-4b7d-9d2f-5f2d4a8c9012",
    "name": "Ашан",
    "logoUrl": "https://{домен}/api/v1/files/store-logos/auchan.png",
    "externalUrl": "https://auchan.example.com",
    "delivery": {
        "type": "time_slot",
        "label": "Ближайшая доставка",
        "valueText": "сегодня 18:00-20:00"
    }
},
{
    "id": "a9c4f2b1-0c5e-4a7d-8b7f-2e9d1a6c3f45",
    "name": "ВкусВилл",
    "logoUrl": "https://{домен}/api/v1/files/store-logos/vkusvill.png",
    "externalUrl": "https://vkusvill.example.com",
    "delivery": {
        "type": "fast_delivery",
        "label": "Быстрая доставка",
        "valueText": "от 20 до 60 минут"
    }
},
{
    "id": "c6e8b4d2-9f1a-4e3c-8d7b-1a2f3c4d5e6b",
    "name": "ВИКТОРИЯ",
    "logoUrl": "https://{домен}/api/v1/files/store-logos/victoria.png",
    "externalUrl": "https://victoria.example.com",
    "delivery": {
        "type": "time_slot",
        "label": "Ближайшая доставка",
        "valueText": "завтра 12:00-14:00"
    }
}
]
}

```

► Ответы на ошибки

Поля ответа:

1. code - string .
2. message - string .

Возможные ошибки

HTTP status: 400 Bad Request - Ошибка возвращается, если запрос составлен некорректно.

Возможные причины:

- не передан обязательный параметр cityId ;

- передан неизвестный `cityId` ;
- параметры `limit` или `offset` переданы в неверном формате;
- значение `limit` меньше 1;
- значение `offset` меньше 0.

Пример ответа, если не передан `cityId` :

```
{
  "code": "VALIDATION_ERROR",
  "message": "Не передан обязательный параметр cityId"
}
```

Пример ответа, если передан неизвестный город:

```
{
  "code": "CITY_NOT_FOUND",
  "message": "Город с указанным идентификатором не найден"
}
```

Пример ответа, если неверно передан `limit`:

```
{
  "code": "VALIDATION_ERROR",
  "message": "Параметр limit должен быть положительным целым числом"
}
```

HTTP status: 500 Internal Server Error - Ошибка возвращается при внутренней ошибке сервера.

Пример ответа:

```
{
  "code": "INTERNAL_SERVER_ERROR",
  "message": "Внутренняя ошибка сервера"
}
```

4. Возможное расширение API

В текущем примере API спроектирован под отображение данных на экране и не включает сортировку или фильтрацию по доставке.

Если в дальнейшем потребуется сортировать магазины по ближайшему времени доставки, фильтровать по типу доставки или проверять актуальность интервала на стороне мобильного приложения, в объект `delivery` можно добавить машиночитаемые поля:

- `startsAt` - дата и время начала ближайшего интервала доставки в формате ISO 8601, например "2026-06-28T21:00:00+03:00";
- `endsAt` - дата и время окончания ближайшего интервала доставки в формате ISO 8601, например "2026-06-28T23:00:00+03:00";
- `durationFromMinutes` - минимальное время быстрой доставки в минутах, например 20 ;
- `durationToMinutes` - максимальное время быстрой доставки в минутах, например 60 .

Также в реальном приложении для более точного расчета доступных магазинов и интервалов доставки можно использовать не только `cityId` , но и точный адрес доставки, район города, зону доставки или геолокацию пользователя.

Задание 3: Архитектура

Описание:

Заказчик хочет, что в мобильное приложение интернет-магазина "Петрушка Зеленая" начали приходить пуши. Они могут быть разными: о том, что заказ слишком долго лежит без действий в корзине, об отмене заказа, рекламные рассылки и прочее. То есть нужен функционал, а какие пуши отправлять - точно найдется.

Что нужно сделать:

Построить верхнеуровневую архитектурную схему - как должна работать отправка PUSH уведомлений в данном приложении. Можно просто в виде блок-схем. Считаем, что на бэкенде микросервисная архитектура. В данном задании рекомендуется в интернете изучить архитектуры подобных решений. Быть готовым на собеседовании обсудить эту схему.

Решение

1. Описание и принятые допущения

Для корректной отправки push backend должен знать, на какое устройство пользователя нужно отправить уведомление. Для этого мобильное приложение сначала получает push-токен устройства и передает его на backend. В связи с этим решение разделено на две схемы:

1. Регистрация push-токена устройства;
2. Отправка PUSH-уведомления по событию из микросервисов.

В рамках данного решения предполагается, что backend построен по микросервисной архитектуре. Отдельные бизнес-сервисы публикуют события, которые могут стать основанием для отправки уведомления. Для упрощения предполагается, что отложенные и периодические события могут публиковаться самими бизнес-сервисами, так как внутри них есть собственный механизм планирования задач. Например, сервис корзины может периодически проверять неактивные корзины и публиковать отложенные или периодические события для отправки напоминания.

Бизнес-сервисы не отправляют PUSH-уведомления напрямую. За подготовку и отправку уведомлений так же отвечает отдельный микросервис уведомлений.

Брокер сообщений используется для асинхронной передачи событий от бизнес-сервисов в сервис уведомлений. Это позволяет не связывать бизнес-сервисы напрямую с логикой отправки PUSH-уведомлений, повышает надежность обработки событий при сбоях или высокой нагрузке.

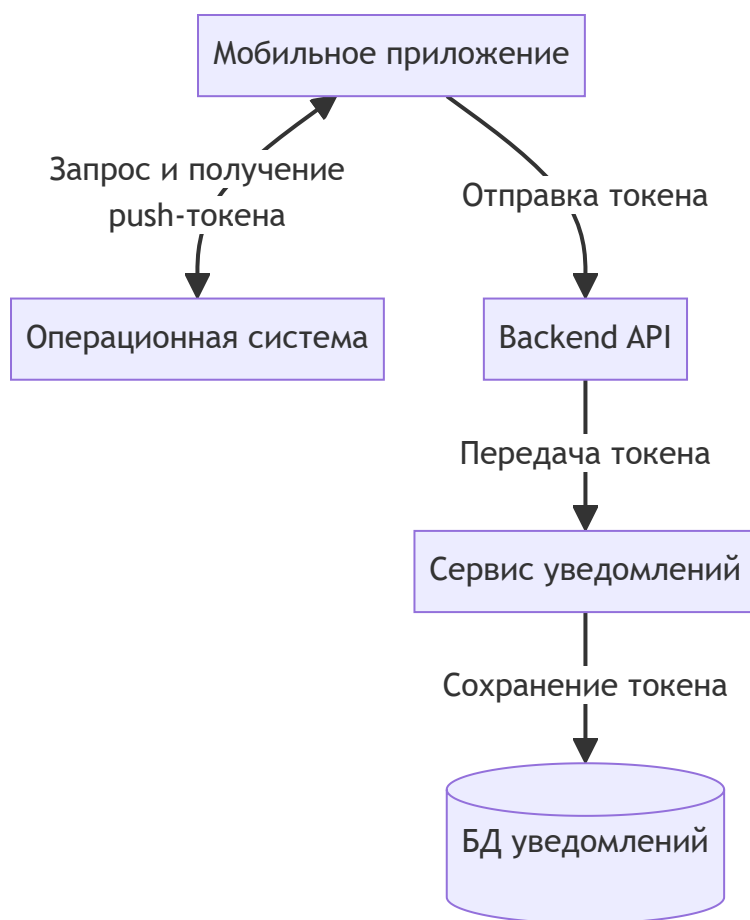
В базе данных уведомлений могут храниться:

- push-токены устройств
- шаблоны уведомлений
- настройки пользователя
- история и статусы отправок

Провайдер PUSH-уведомлений отвечает за доставку уведомления на устройство пользователя. В реальном приложении для Android может использоваться FCM, для iOS — APNs и тд.

2. Схемы

2.1. Регистрация push-токена устройства

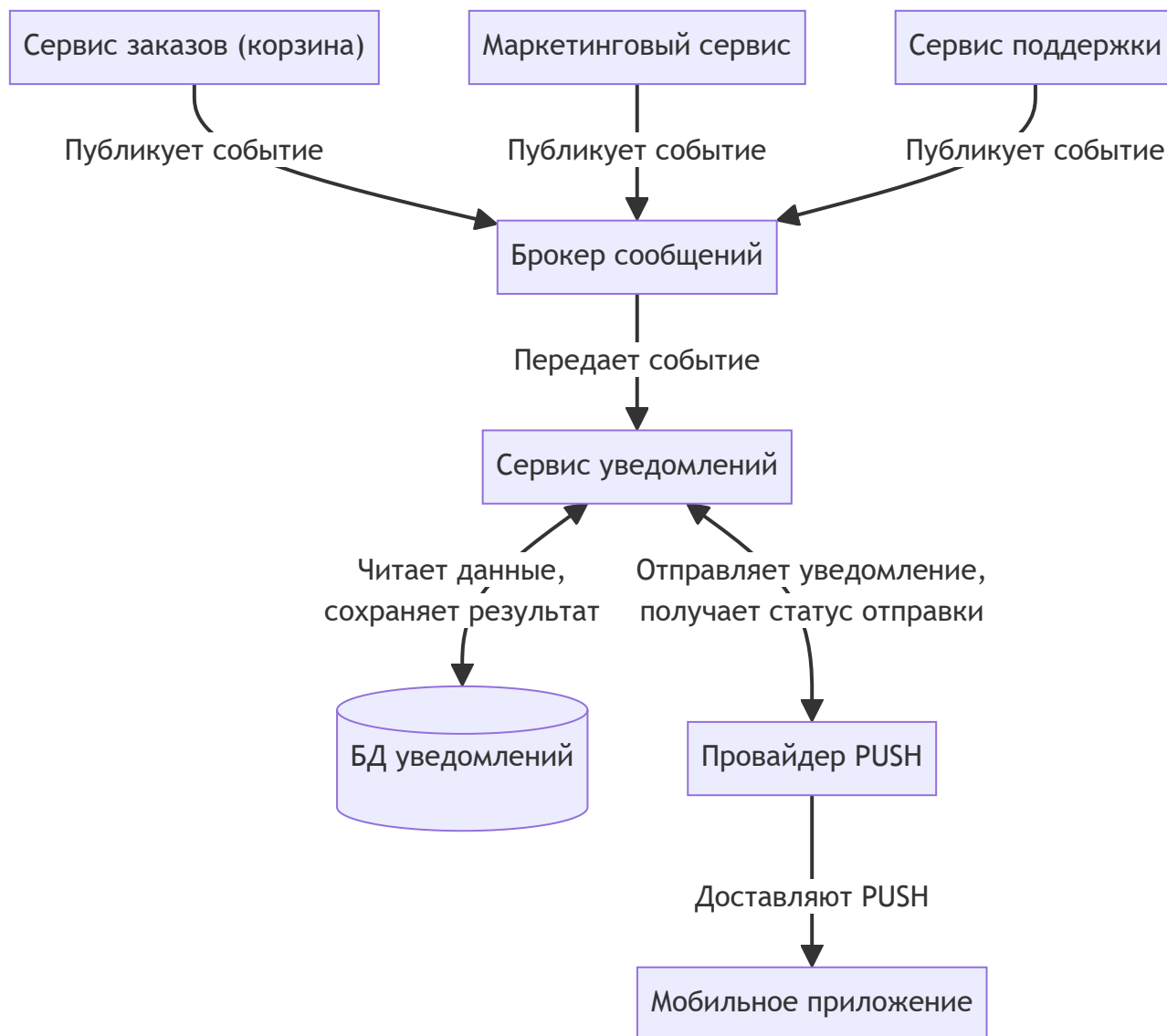


Описание схемы

Перед отправкой PUSH-уведомлений мобильное приложение должно получить push-токен устройства. Push-токен нужен для того, чтобы backend мог определить, на какое устройство пользователя отправлять уведомление. Если пользователь авторизован на нескольких устройствах, у него может быть несколько push-токенов. В этом случае сервис уведомлений может хранить несколько токенов для одного пользователя.

Мобильное приложение получает push-токен через операционную систему и механизм push-уведомлений. После этого приложение отправляет токен на backend (например Rest API методом: POST /push-tokens). Backend передает полученный токен в сервис уведомлений. Сервис уведомлений сохраняет или обновляет токен в базе данных уведомлений.

2.2. Отправка PUSH-уведомления



Описание схемы

Сервисы публикуют события в брокер сообщений. Событие содержит данные, которые нужны для подготовки уведомления:

- тип события
- идентификатор пользователя
- дополнительные параметры для текста уведомления и тд.

Сервис уведомлений получает событие из брокера сообщений и выполняет основную логику отправки:

- определяет тип уведомления;
- получает шаблон уведомления;
- проверяет настройки пользователя;
- получает push-токен устройства;
- формирует текст уведомления;
- отправляет уведомление через push-провайдера;
- сохраняет результат отправки в базе данных уведомлений.

Провайдер PUSH-уведомлений доставляет уведомление в мобильное приложение. Сервис уведомлений может получить от провайдера статус отправки, например успешная отправка, ошибка или невалидный push-токен.

Если push-токен стал невалидным, сервис уведомлений может пометить его как неактивный, чтобы не использовать его для следующих отправок.